

```
=====
Python Data Structures
=====
```

**** Data structures are ways to store, organize, and manage data in a program.**

**** In Python, the most commonly used built-in data structures are:**

- 1) List
- 2) Tuple
- 3) Set
- 4) Dictionary
- 5) String

Note: Each data structure has a different purpose.

```
=====
## What is a List?
=====
```

**** A list is a built-in data structure in Python used to store multiple values in a single variable.**

Syntax : my_list = [value1, value2, value3]

Examples :

marks = [80, 90, 75]

students = ["Ravi", "Sita", "Kiran"]

mixed_data = ["Ravi", 25, 85.5, True]

**** Ordered**

**** Mutable / Changeable**

**** Allow Duplicate Values**

**** Can store different data types of data (homogenous & heterogenous)**

**** We can access list elements using index order**

**** in python index starts from 0**

list creation

courses = ["JAVA", "Python", "DevOps", "GEN AI"]

access list elements

print(courses[0])

print(courses[1])

print(courses[2])

print(courses[3])

items = ["Laptop", "Mouse", "Keyboard", "Monitor"]

Positive Indexing (starts from left to right)

print(items[0])

```
#negative index (starts from right to left)
print(items[-1])
```

```
# List Slicing :: used to get a part of list
# syntax : list_name[start:end]
```

```
numbers = [10, 20, 30, 40, 50, 60]
print(numbers) # print all elements
```

```
print(numbers[1:4]) #start index inclusive, end index exclusive
```

```
print(numbers[:3]) # print from 0th index to 2nd index
```

```
print(numbers[2:]) # print from 2nd to last
```

```
print(numbers[::2]) # step by 2
```

```
print(numbers[::-1]) # print in reverse order
```

```
#####
# List Operations
#####
```

```
# append : append() adds an element at the end of the list
# insert : insert() adds an element at specified index
# extends : extend() adds multiple elements to the list.
# remove : remove() removes the specified value
# pop : pop() removes element based on index. If index is not given, pop() removes the last element
# clear : clear() removes all elements from the list.
# del : del can delete an element or entire list.
```

```
courses = ["Python", "Java"]
courses.append("DevOps")
print(courses)
```

```
courses.insert(1, "GEN AI")
print(courses)
```

```
frontend = ["HTML", "CSS"]
backend = ["Python", "Django"]
frontend.extend(backend)
print(frontend)
```

```
courses.remove("Java")
print(courses)
```

```
courses.pop()
print(courses)
```

```
courses.clear()
print(courses)
```

```
courses = ["Python", "Java", "DevOps"]
del courses[1]
print(courses)
```

```
##### List functions #####
```

```
print(len(courses))
```

```
numbers = [1, 3, 6, 9, 2]
```

```
print(len(numbers))
```

```
print(min(numbers))
```

```
print(max(numbers))
```

```
print(sum(numbers))
```

```
numbers.sort()
```

```
print(numbers)
```

```
numbers.sort(reverse=True)
```

```
print(numbers)
```

```
#####
```

```
=====
```

```
# List Comprehension in Python
```

```
=====
```

** List comprehension is a short and clean way to create a new list from an existing sequence like range, list, tuple, or string.

** It is mainly used to reduce multiple lines of loop code into a single line.

```
# normal way
```

```
numbers = []
```

```
for i in range(1,6):
```

```
    numbers.append(i)
```

```
print(numbers)
```

```
# list comprehension
```

```
numbers = [i for i in range(1,6)]
```

```
print(numbers)
```

```
# square of numbers in normal way
```

```
squares = []
```

```
for i in range(1,6):
```

```
    squares.append(i*i)
```

```
print(squares)
```

```
# square of numbers using comprehension
```

```
squares = [i * i for i in range(1,6)]
```

```
print(squares)
```

```
# Even Numbers normal way
```

```
even_numbers = []
```

```
for i in range(1,11):
```

```
    if i % 2 == 0:
```

```
        even_numbers.append(i)
```

```
print(even_numbers)
```

```
# comprehension
```

```
even_numbers = [i for i in range(1,11) if i % 2 ==0]
```

```
print(even_numbers)
```

```
# convert to uppercase
```

```
names = ["ravi", "sita", "kiran"]
for name in names:
    print(name.upper())

upper_names = [name.upper() for name in names]
print(upper_names)

# students grade
marks = [80, 30, 90, 45, 20]
results = ["Pass" if mark>=35 else "Fail" for mark in marks]
print(results)

# GST
prices = [1000, 2000, 5000, 10000]
price_with_gst = [price + (price * 18 /100) for price in prices]
print(price_with_gst)
```

```
=====
## What is a Tuple?
=====
```

**** A tuple is a built-in data structure in Python used to store multiple values in a single variable.**

**** Tuple is similar to List, but the main difference is:**

-- List is mutable

-- Tuple is immutable / not changeable

**** Ordered**

**** Immutable**

**** Allow duplicate values**

**** Can store different data types**

**** Faster than list**

**** Used to store fixed data**

Syntax : my_tuple = (value-1, value-2, value-3)

```
courses = ("JAVA", "Python", "GEN AI", "DEVOPS", ".Net", "MERN")
print(courses)
```

```
print(courses[0])
print(courses[1])
print(courses[-1])
```

```
print(courses[1:3])
```

```
print(courses[-1:-3:-1]) #start is inclusive, end is exclusive
print(courses[:3])
print(courses[2:])
```

```
## courses[1] = "AWS" (not possible)
```

```
print(courses.index("GEN AI"))

# tuple packing
student = "Ashok", "Male", 66868686
print(student)

# tuple unpacking
student = ("Ashok", "Male", 66868686)
name, gender, phone = student
print(name)
print(gender)
print(phone)

## Tuple functions

# len()
# max ()
# min ()
# sum ()
# sorted ()

numbers = (10, 15, 30, 20, 40, 25)
print(len(numbers))
print(sum(numbers))
print(min(numbers))
print(max(numbers))

### numbers.sorted() (invalid)

# sorted() returns list, not tuple.

sorted_numbers = sorted(numbers) #it returns new list
print(sorted_numbers)

sorted_numbers = sorted(numbers, reverse=True) #it returns new list
print(sorted_numbers)

courses = ("Python", "Java", "DevOps")

# convert tuple to list
course_list = list(courses) # it gives list
course_list[0] = "GEN AI"
print(course_list)

## Check Value Exists in Tuple

if "GEN AI" in courses:
    print("GEN AI course available in Ashok IT")
else:
    print("GEN AI course not available in Ashok IT")

# Tuple concatnation
frontend = ("HTML", "CSS")
backend = ("Python", "Django")
fullstack = frontend + backend
print(fullstack)

skills_list = list(fullstack)
skills_list.insert(-1,"DevOps")
print(skills_list)
```

```
=====
## Difference Between List and Tuple
=====
```

List is mutable

List uses square brackets []

List is slower than tuple

List is used when data changes frequently

Tuple is immutable

Tuple uses parentheses ()

Tuple is faster than list

Tuple is used when data is fixed

```
=====
## What is a Set?
=====
```

**** A set is a built-in data structure in Python used to store multiple values in a single variable.**

**** Set is mainly used when we want to store unique values.**

**** Set does not allow duplicate values.**

Set Features

**** It is un-ordered**

**** Mutable**

**** Doesnot allow duplicates**

**** Can store different data types**

**** Doesn't support indexing**

**** Doesn't support slicing**

syntax : my_set = {value-1, value-2, value-3}

courses = {"Python", "JAVA", "Python", "DEVOPS"}

print(courses)

How data is storing internally (how duplicates are removed)

Python â†’ hash calculated â†’ stored

JAVA â†’ hash calculated â†’ stored

Python â†’ hash calculated â†’ already exists, so ignored

DEVOPS â†’ hash calculated â†’ stored

```
##### Example #####
```

```
courses = {"Python", "JAVA", "DEVOPS", "Python"}
print(hash("Python"))
print(hash("JAVA"))
print(hash("JAVA"))
```

```
#### Set Operations ####
```

```
# add ( ) : It is used Add one element to set
# update ( ) : It is used Add multiple elements to set
# remove ( ) : It is used to remove specified element from set
# discard ( ) : It is used to remove specified element from set
# pop ( ) : It is used to remove random element from set
# clear ( ) : It is used to remove all elements from set
# del : It is used to delete entire set from memory
```

```
courses.add("GEN AI")
print(courses)
```

```
courses.update(["GCP", "AWS", "aws", "Azure"])
print(courses)
```

```
#courses.remove("HTML")
print(courses)
```

```
# If the element is not available, discard() does not give error.
```

```
courses.discard("AWS")
print(courses)
```

```
courses.pop()
print(courses)
```

```
courses.clear()
print(courses)
```

```
del courses
## print(courses)
```

```
## Set Functions
```

```
# len()
# min()
# max()
# sorted()
```

```
numbers = {10, 50, 20, 80, 30, 40}
print(len(numbers))
print(min(numbers))
print(max(numbers))
print(sum(numbers))
sorted_numbers = sorted(numbers, reverse=True)
print(sorted_numbers)
```

```
# Remove Duplicates from list using set() function
skills = ["JAVA", "Python", "DevOps", "JAVA"]
print(set(skills))
```

```
=====
## Difference Between List,Tuple and Set
```

```
=====
```

```
-- List is mutable
-- List uses square brackets []
-- List is ordered
-- List allows duplicates
-- List is slower than tuple
-- List is used when data changes frequently

-- Tuple is immutable
-- Tuple uses parentheses ()
-- Tuple is ordered
-- Tuple allows duplicates
-- Tuple is faster than list
-- Tuple is used when data is fixed

-- Set is Mutable
-- Set uses { }
-- Set is un-ordered
-- Set don't allow duplicates
-- Set not supporting for indexing and slicing
```

```
=====
```

```
## What is a Dictionary?
```

```
=====
```

```
** A dictionary is a built-in data structure in Python used to store data in key-value pairs.
```

```
** Dictionary is used when we want to store data with a meaningful name/key.
```

```
## Dictionary Features
```

```
** Ordered
```

```
** Mutable
```

```
** Duplicate keys not allowed, Duplicate values allowed
```

```
** Storing data in key-value pair
```

```
** Keys must be unique
```

```
** Value can be any data type
```

```
Syntax ::
```

```
my_dict = {
    "key-1" : "value-1",
    "key-2" : "value-2"
}
```

```
Example :
```

```
student = {
    "name": "Ravi",
    "age": 25,
    "course": "Python"
}
```

```
-----
```



```
student = {
    "name": "Ravi",
    "age": 25,
    "course": "Python",
    "marks": 85
}
print(student)
print(student["name"]) # if key not present it gives error
print(student.get("gender")) # if key not present it gives None
print(student.get("city", "Hyd")) # If key not present gives default val

## Adding New Key-Value Pair
student["grade"] = "A"
print(student)

student["name"] = "Raj"
print(student)

# Dictionary Operations
print(student.keys())
print(student.values())
print(student.items())

# update() is used to update dictionary values or add new key-value pairs.
student.update({"name": "Ashok"})
print(student)

# pop() removes value based on key.
student.pop("grade")
print(student)

# popItem() removes last inserted key-value pair
student.popitem()
print(student)

# clear : removes all key-value pairs from dictionary
# del : delete entire dictionary from memory

del student["name"]
print(student)

del student

# Working wit Nested Dictionary

students = {
    "s1": {
        "name": "Raj",
        "age": 25,
        "course": "Python"
    },
    "s2": {
        "name": "Anil",
        "age": 35,
        "course": "GEN AI"
    }
}
print(students)
print(students["s1"])
print(students["s1"]["name"])
print(students["s1"].get("name"))
```

```
print(students["s2"])
print(students["s2"]["name"])

# Looping Nested Dictionary
for student_id, student_data in students.items():
    for key,value in student_data.items():
        print(key, "--", value)

    print("-----")

=====
## What is a String?
=====

==> A string is a built-in data type in Python used to store text data.

==> String is a sequence of characters.

==> Characters can be letters, numbers, symbols, or spaces.

Ex:
    name = "Ashok"

    pan_number = "DB7979DDF"

    pwd = "Ashok@685757"

## String Features

** Ordered data structure

** Immutable / Not changeable

** Supports Indexing

** Supports Slicing

** Allow Duplicate Characters

** Can be created using single quotes, double quotes, or triple quotes

## String Creation

    name = 'Ashok'

    msg = "Hello, how are you?"

    text = """
        I am learning python
        python is easy
    """

# String operations

name = "Python"
print(len(name))

# index based operations
print(name[0])
print(name[-1])
```

```
# slicing operations
print(name[0:3]) # Pyt
print(name[3:]) #hon

# name[0] = "J" # Error

f_name = "Ashok"
l_name = "Bollepalli"

full_name = f_name + " " + l_name
print(full_name)

print(full_name * 3)

msg = "i like gen ai development"
print(msg.title())
print(msg.capitalize())

# Check character or word exists (True/False)
course = "GEN AI and Agentic AI"
print("AI" in course)
print("Python" in course)

# String with forloop
for letter in course:
    print(letter)

# String methods

# upper() : convert all characters to upper case
# lower() : convert all characters to lower case
# title() : First character of every word converts into upper case
# capitalize() : converts first letter of string into uppercase.
# strip() : removes spaces from beginning and ending of string.
# lstrip() : lstrip() removes spaces from left side.
#rstrip() :rstrip() removes spaces from right side.
# replace() : Replace old value with new value and creates new string
# split() : splits string into list of elements
# join() : joins list values into string (we can give delim)
# find() : returns index position of given value.
# count() : returns how many times value is repeated
# startsWith() : checks whether string starts with given value.
# endsWith() : checks whether string ends with given value.
# isdigit() : checks whether all characters are digits.
# isalpha() : checks whether all characters are alphabets.
# isalnum() : checks whether all characters are alphabets or numbers.
# format() : Construct a string with dynamic values

message = "I like Java"
new_msg = message.replace("Java","Python")
print(new_msg)

courses = "Python,Java,DevOps,AWS"
result = courses.split(",")
print(result)

courses = ["Python", "Java", "DevOps"]
result = "#".join(courses)
print(result)
```

```
str = "welcome to c and python"
print(str.find('c')) # 3
print(str.find('z')) # -1
print(str.find('python')) # 11

print(str.index('python')) # 11
## print(str.index('z')) # Error

message = "Python is easy and Python is powerful"
print(message.count("Python"))

print(message.startswith("Python"))
print(message.endswith("Python"))

mobile = "85858658"
print(mobile.isdigit())

name = "AshokIT"
print(name.isalpha())

pwd = "India@123"
print(pwd.isalnum())

# String formatting is used to insert variable values inside a string.
name = "Ashok"
course = "Python"

print("Student name is", name, "and course is", course)
print("Student name is " + name + " and course is " + course)

msg = "Student name is {} and course is {}".format(name, course)
print(msg)

# f-string
msg = f"Student name is {name} and course is {course}"
print(msg)

# usecase : sql query to verify user entered login credentials
# Ex : f"select * from user_tbl where email={email} and pwd={pwd}"

# Reverse a string

text = "python"
rev_text = reversed(text)
print(rev_text)
print(list(rev_text))
print("".join(reversed(text)))

# Email Validation
email = input("Enter Email")
if "@" in email and "." in email:
    print("Valid Email")
else:
    print("Invalid Email")

# Generate Username based on name and last 4 digits of phno
```

```
name = "Ashok"  
phno = "868686868"  
username = name.lower()+ phno[-4:]  
print(username)
```

Practice programs

- 1) Reverse a string
- 2) Check Given String is palindrome or not
- 3) Count vowels and consonants in string
- 4) Count occurrence of each character in given string
- 5) Remove Duplicate Characters in String
- 6) Convert String into upper case without using upper()
- 7) Convert String into lower case without using lower()
- 8) Find First Non-Repeated Character in String
- 9) Find Most Repeated Character in String
- 10) Check Anagram Strings
- 11) Count digits, letters and special characters in given string
- 12) Reverse Each word in a sentence
- 13) Find Longest word in sentence
- 14) Count Frequency of Each word in string